

12 — Probabilistic models 2

1

Machine Learning (MPA-MLR)

Roman Jakubíček

Department of Biomedical Engineering · FEEC, Brno University of Technology

2025

0 • Why probability at all

Loss functions, regularization and generative models are probability in disguise.

1 • Naive Bayes classifier

One d -dimensional density replaced by $K \times d$ one-dimensional ones.

2 • Gaussian mixture model

Several Gaussians instead of one — and the EM algorithm that fits them.

3 • Logistic regression

A sigmoid, a Bernoulli likelihood, and the cross-entropy we already know.

Previous lecture — Probabilistic models 1

- Probability introduction + Bayes' rule
- Probabilistic models — parameter estimation:
 - Maximum likelihood (MLE)
 - Maximum a-posteriori (MAP)
 - Bayesian inference

This lecture — Probabilistic models 2

- Probabilistic models — example applications:
 - Naive Bayes classifier
 - Gaussian mixture model — EM algorithm
 - Logistic regression

Last time we asked *what are the parameters of one distribution?* Today we ask *what can we build out of such distributions?*

Why probability theory is fundamental — losses and regularization

A lot of things in machine learning are **derived** using probability.

- **Loss functions = assumptions about noise, plus MLE.** The L2 loss from the sixth lecture is the maximum likelihood solution under Gaussian noise; cross-entropy is the maximum likelihood solution under a Bernoulli (or categorical) likelihood. We derive both today.
- **Regularization = a prior.** Weight decay from the sixth lecture is exactly a zero-mean Gaussian prior on the weights inside a MAP estimate; DropOut switches neurons off with a Bernoulli variable.

$$\underbrace{\arg \min_{\mathbf{w}} \sum_i L(y_i, f_{\mathbf{w}}(\mathbf{x}_i)) + \lambda \|\mathbf{w}\|^2}_{\text{what we have been writing since the fourth lecture}} = \underbrace{\arg \max_{\mathbf{w}} p(D | \mathbf{w}) p(\mathbf{w})}_{\text{MAP from the previous lecture}}$$

Why probability theory is fundamental — optimization and generative models

- **Stochastic gradient descent = Monte Carlo estimation.** The mini-batch gradient from the sixth lecture is an *unbiased* estimate of the full-data gradient — that is exactly what a Monte Carlo estimate is.
- **Generative models = modelling the data distribution**, compared with a KL divergence (GANs, VAEs, diffusion models, normalizing flows, autoregressive models). The generative-versus-discriminative table from the previous lecture is the same distinction.
- **Transformers and language models = conditional probabilities.** The transformer decoder from the tenth lecture predicts the next token — that is $p(x_t \mid x_1, \dots, x_{t-1})$, nothing else.

Probability theory is the theoretical engine behind innovation in machine learning — essential for discovering new models, even if it is hidden during the practical implementation.

Example — regression: the model and the likelihood

6

Model (prediction plus Gaussian noise):

$$y_i = f_{\boldsymbol{\theta}}(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Likelihood of one sample — the noise distribution shifted to the prediction:

$$p(y_i \mid x_i, \boldsymbol{\theta}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f_{\boldsymbol{\theta}}(x_i))^2}{2\sigma^2}\right)$$

Likelihood of the whole i.i.d. data set $D = \{(x_i, y_i)\}_{i=1}^n$:

$$L(\boldsymbol{\theta}) = p(D \mid \boldsymbol{\theta}) = \prod_{i=1}^n p(y_i \mid x_i, \boldsymbol{\theta})$$

The original slide jumps straight from one sample to the sum; the product step is added here. The original also typesets the normal distribution with a Greek capital N — it is $\mathcal{N}$, as in the previous lecture.

Example — regression: from the negative log-likelihood to the L2 loss

7

Negative log-likelihood — the product becomes a sum:

$$-\log L(\boldsymbol{\theta}) = \sum_{i=1}^n \frac{(y_i - f_{\boldsymbol{\theta}}(x_i))^2}{2\sigma^2} + \underbrace{n \log \sqrt{2\pi\sigma^2}}_{\text{const}}$$

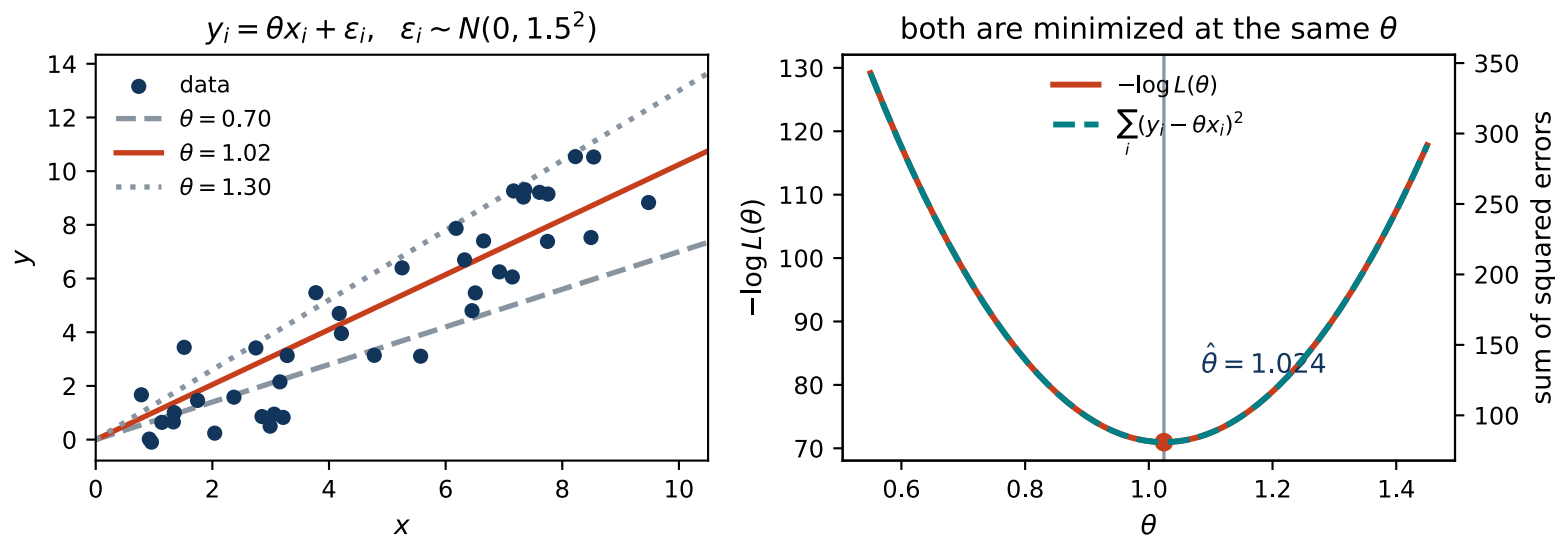
Minimization — constants and positive factors do not move the minimum:

$$\boldsymbol{\theta}^* = \arg \min_{\boldsymbol{\theta}} \sum_{i=1}^n (y_i - f_{\boldsymbol{\theta}}(x_i))^2$$

This is the **L2 loss from the sixth lecture**. It is not a definition — it is what MLE gives us once we assume Gaussian noise.

The same minimum — computed

8



40 samples of $y_i = \theta x_i + \varepsilon_i$ with $\varepsilon_i \sim \mathcal{N}(0, 1.5^2)$. The two curves on the right differ only by an additive constant and a positive factor, so they are minimized at the same $\hat{\theta} = 1.024$ — which is also the least-squares solution $\hat{\theta} = \sum_i x_i y_i / \sum_i x_i^2$.

1 · Naive Bayes classifiers

How do we turn distributions into a classifier — and what do we gain by pretending the features are independent?

Given measured data $\mathbf{x}$ and a class $y \in \{y_1, \dots, y_K\}$, we want the **posterior probability of the class** $p(y_k \mid \mathbf{x})$. We apply Bayes' rule:

$$\underbrace{p(y_k \mid \mathbf{x})}_{\text{posterior}} = \frac{\overbrace{p(\mathbf{x} \mid y_k)}^{\text{likelihood}} \overbrace{p(y_k)}^{\text{prior}}}{\underbrace{p(\mathbf{x})}_{\text{evidence}}} = \frac{p(x_1, x_2, \dots, x_d \mid y_k) p(y_k)}{\sum_{j=1}^K p(x_1, x_2, \dots, x_d \mid y_j) p(y_j)}$$

- $p(\mathbf{x} \mid y_k)$ — a **conditional density**, estimated from the training samples of class k .
- $p(y_k)$ — estimated from the data as the **frequency** of class k .
- $p(\mathbf{x})$ — a **sum over K discrete classes**, so it can always be computed. It only normalizes the posterior to 1.

The original writes the summation index in the denominator as k , the same letter that is fixed in the numerator; the slide on the EM algorithm writes it correctly as $\sum_{j=1}^K$ and that form is used here.

Bayesian classifiers — the decision rule and its problem

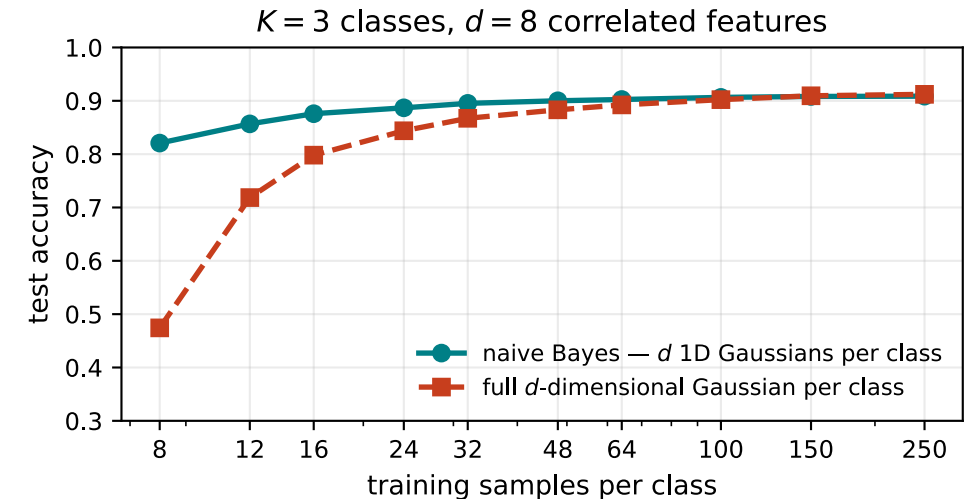
11

We are most interested in the **most probable class**. The denominator does not depend on k , so it drops out:

$$y_k^* = \arg \max_{y_k} p(\mathbf{x} \mid y_k) p(y_k)$$

From the training set D we take only the samples of class k and fit the parameters of $p_{\theta}(x_1, \dots, x_d \mid y_k)$, e.g. by ML estimation.

However — this distribution is d -dimensional. A full Gaussian needs $d + \frac{d(d+1)}{2}$ parameters **per class**:
65 numbers for $d = 10$, 5150 for $d = 100$.



A **simulation** — the seminar data has only $d = 2$, so it cannot show what happens in eight dimensions. Averaged over 40 repetitions.

Assume the features are **mutually independent, conditional on the class** y_k :

$$y_k^* = \arg \max_{y_k} p(x_1, \dots, x_d \mid y_k) p(y_k) = \arg \max_{y_k} p(y_k) \prod_{i=1}^d p_{\theta}(x_i \mid y_k)$$

One d -dimensional density is replaced by a **product of d one-dimensional densities** — and each of them may be of a different type (normal for a continuous feature, Bernoulli for a binary one, categorical for a discrete one).

Independence *conditional on the class* is much weaker than plain independence. In the seminar data set tumour size and HPV status have correlation 0.16 overall, but only 0.03 / 0.05 / 0.09 **inside** the three classes — so the naive assumption is nearly satisfied here, even though the features look dependent when the classes are pooled.

The task is to find the distributions $p(x_i \mid y_k)$, each described by a parameter vector $\theta_{i,k}$ for the i -th feature conditioned on the k -th class — that is $K \times d$ **distributions**.

For the i -th feature we have observed n values $[f_1, f_2, \dots, f_n]$; the parameters $\theta_{i,k}$ are estimated only from the subset $\mathbf{f}_{i,k} = \mathbf{f}(x_i \mid y_k)$ belonging to class k , because $p(x_i \mid y_k)$ is conditioned on y_k :

$$\theta_{\text{MAP}}^* = \arg \max_{\theta_{i,k}} p(\theta_{i,k} \mid \mathbf{f}_{i,k}) = \arg \max_{\theta_{i,k}} \frac{p(\mathbf{f}_{i,k} \mid \theta_{i,k}) p(\theta_{i,k})}{p(\mathbf{f}_{i,k})}$$

Drop the prior $p(\theta_{i,k})$ and this is plain MLE — the estimates we derived in the previous lecture.

The original writes „ $k \times d$ distributions“ with a lower-case k , although K is the number of classes; and it mixes $\theta_{i,k}$ with $\theta_{i,k}$ in two neighbouring lines. Both are unified here.

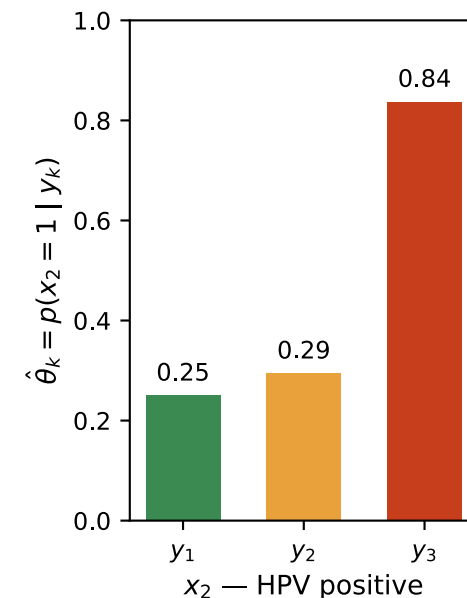
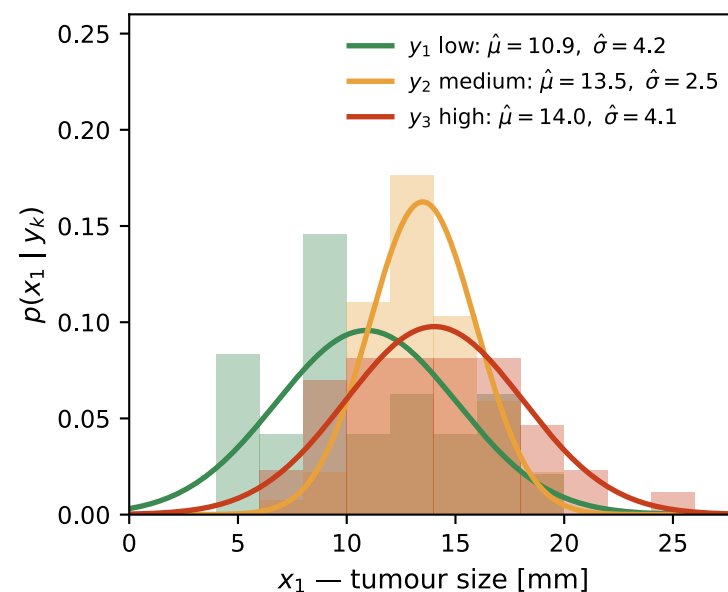
Naive Bayes — a worked example

14

The data set of the **seminar** — 203 patients, $K = 3$ classes (tumour aggressiveness), $d = 2$ features:

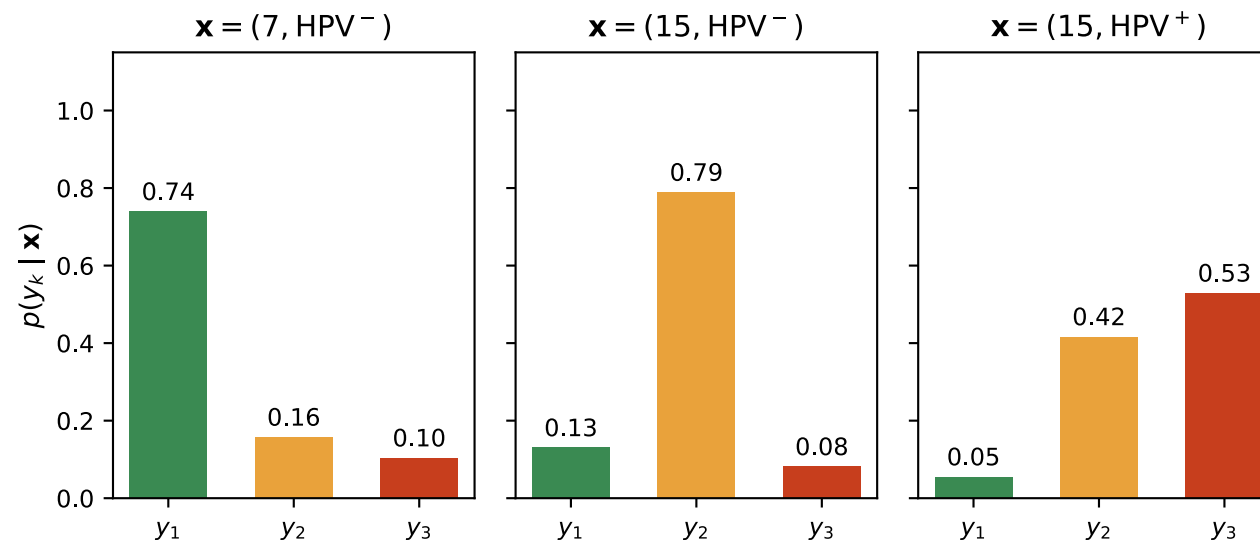
- x_1 — tumour size, **normal** $\mathcal{N}(\mu_k, \sigma_k^2)$,
- x_2 — HPV positivity, **Bernoulli** $\theta_k^{x_2} (1 - \theta_k)^{1-x_2}$.

Six one-dimensional distributions (nine parameters) plus three class priors — all from plain MLE on a two-thirds training split. Test accuracy 0.647, against 0.485 for always answering the majority class.



Naive Bayes — the posteriors add up to one

15



$\mathbf{x}$	$p(y_1 \mathbf{x})$	$p(y_2 \mathbf{x})$	$p(y_3 \mathbf{x})$
7 mm, HPV—	0.739	0.157	0.104
15 mm, HPV—	0.130	0.789	0.081
15 mm, HPV+	0.055	0.416	0.529

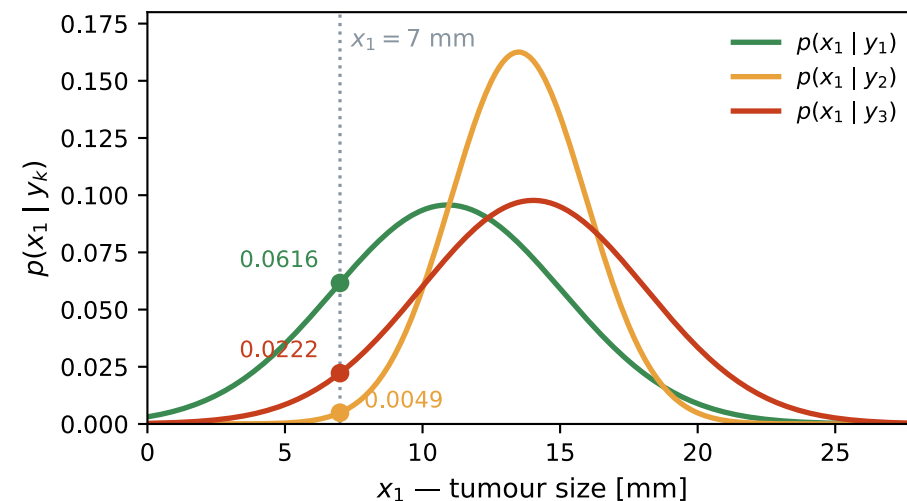
These are the three samples of the seminar. The same size, a different HPV status — and the **most probable class moves from y_2 to y_3** , because a single factor in the product changes from $1 - \theta_k$ to θ_k .

Naive Bayes — one posterior by hand

The first sample, $\mathbf{x} = (7 \text{ mm}, \text{HPV}^-)$: three factors per class, multiply, normalize.

k	$p(y_k)$	$p(x_1 = 7 y_k)$	$p(x_2 = 0 y_k) = 1 - \theta_k$	product	$p(y_k \mathbf{x})$
1	0.178	0.0616	0.750	0.00822	0.739
2	0.504	0.0049	0.706	0.00175	0.157
3	0.319	0.0222	0.163	0.00115	0.104
				$\Sigma = 0.01112$	1

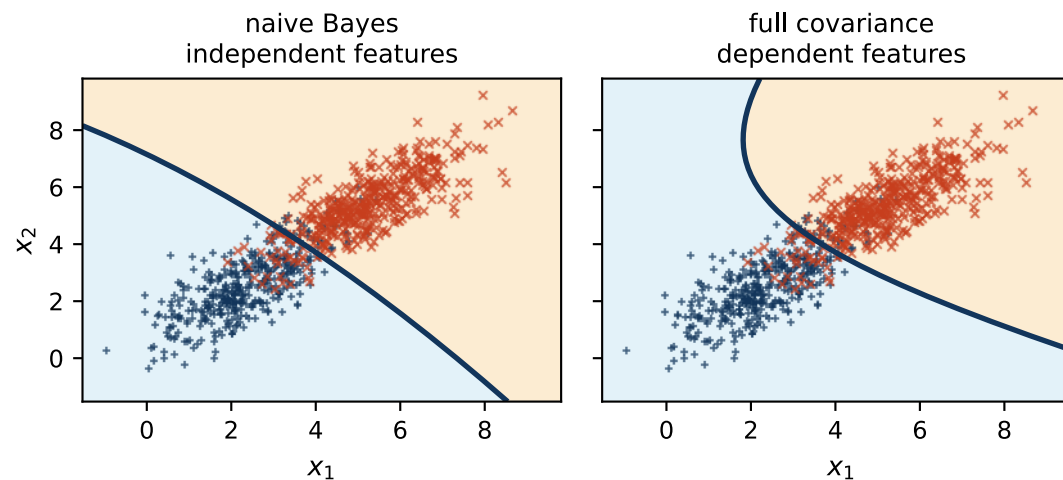
The evidence $p(\mathbf{x}) = 0.01112$ is the sum of the products; each posterior is a product divided by it. The class prior favours y_2 (0.504), but a 7 mm tumour is twelve times less likely under y_2 than under y_1 — and that decides.



The middle factor is the density $\mathcal{N}(7; \hat{\mu}_k, \hat{\sigma}_k^2)$ read off the three curves.

Naive — assume the features are mutually independent:

- we get a **less complex model** — easy to estimate, but...
- **in real-world data it is not true** — features are usually somehow mutually dependent, but...
- **even a wrong and simple model can perform better than a correct and complex one.** Classification only needs the *ranking* of the posteriors to be right, not their values.



A second **simulation**, now $d = 2$ so the boundary can be drawn — two classes with feature correlation 0.77. Accuracy on the same 800 points: 0.904 naive, 0.896 full — [kod/bayesova_hranice.py](#).

Pros

- **Robust to missing features** — only the features we have enter the product.
- **Robust to dependency of features** (see the previous slide).
- **A different distribution can be chosen for each feature.**
- Parameters are estimated by MLE / MAP — easy to compute and interpretable.
- Thanks to the conditional independence we can estimate θ **more accurately with less data** — a product of 1D distributions instead of a complex d -dimensional one.

Cons

- We must **assume the form** of every feature distribution $p(x_i \mid y_k)$ (and a prior on θ if we use MAP).
- The posteriors are over-confident when the features are correlated.

Missing feature, computed for the sample (15 mm, HPV+) — the posteriors over $y_1 \mid y_2 \mid y_3$:

- both features: 0.055 / 0.416 / **0.529**,
- size only: 0.097 / **0.624** / 0.279,
- HPV only: 0.097 / 0.323 / **0.581**.

A posterior comes out either way, with nothing imputed — a *different* one, of course.

2 · Gaussian mixture model

One Gaussian is rarely enough. Several of them are — but then we no longer know which point belongs to which.

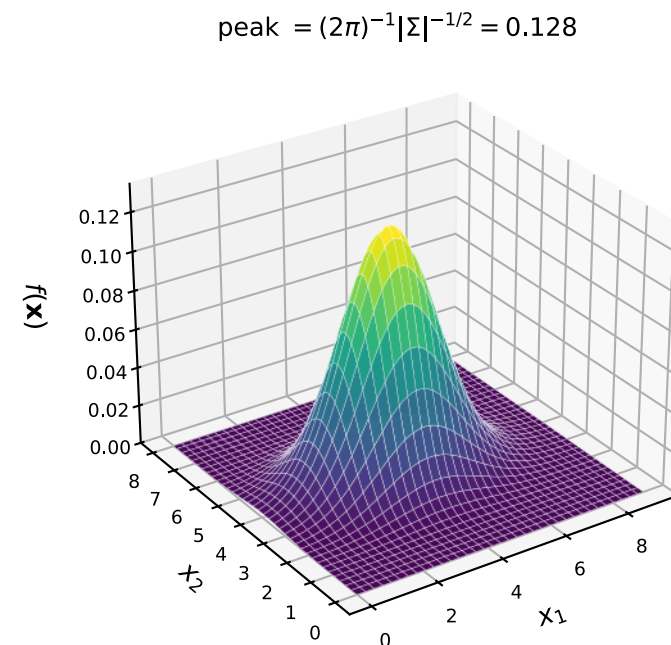
d -dimensional Gaussian function:

$$f(\mathbf{x}) = (2\pi)^{-\frac{d}{2}} |\Sigma|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{x} - \boldsymbol{\mu})\right)$$

with $\boldsymbol{\mu} = (\mu_1, \dots, \mu_d)^T$, $\mathbf{x} = (x_1, \dots, x_d)^T$ and

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \cdots & \sigma_{1d} \\ \vdots & \ddots & \vdots \\ \sigma_{d1} & \cdots & \sigma_d^2 \end{bmatrix}, \quad \sigma_{ij} = \text{cov}(x_i, x_j)$$

For the two-dimensional case: $\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$, $\boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix}$, $\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} \\ \sigma_{21} & \sigma_2^2 \end{bmatrix}$.



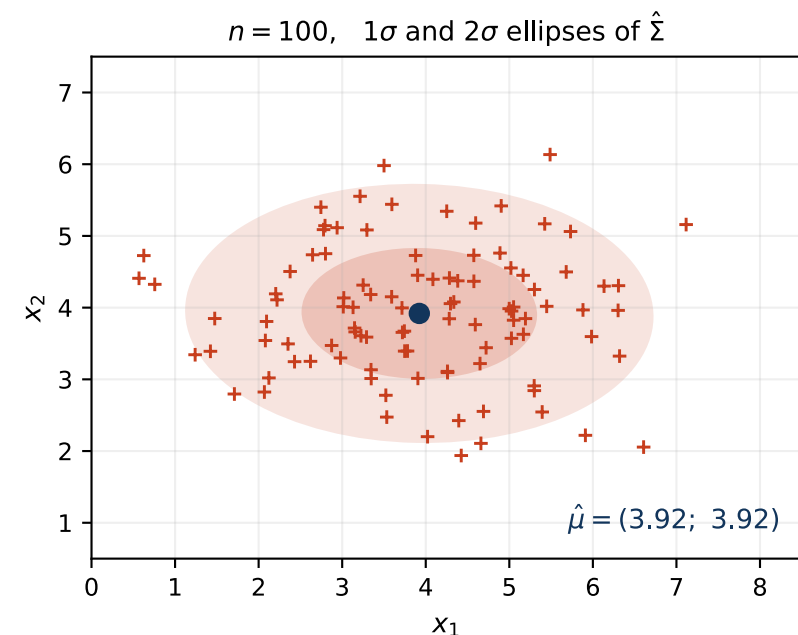
The original writes the off-diagonal entries as products $\sigma_1 \sigma_d$. Such a matrix is $\mathbf{ss}^T$, hence singular, and the same slide needs $|\Sigma|^{-1/2}$ and Σ^{-1} — so they are written here as covariances σ_{ij} . The original 3D surface is not a normalized density (its peak is 0.54); this one is.

Estimation using MLE, exactly as in the previous lecture, only in d dimensions:

$$\boldsymbol{\mu}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i = \bar{\mathbf{x}}$$

$$\boldsymbol{\Sigma}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^n (\mathbf{x}_i - \boldsymbol{\mu}_{\text{MLE}})(\mathbf{x}_i - \boldsymbol{\mu}_{\text{MLE}})^T$$

For $n = 100$ points of the red cluster: $\hat{\boldsymbol{\mu}} = (3.92; 3.92)$ and $\hat{\boldsymbol{\Sigma}} = \begin{bmatrix} 1.94 & -0.03 \\ -0.03 & 0.80 \end{bmatrix}$. The ellipses are drawn from the **eigenvectors** and **eigenvalues** of $\hat{\boldsymbol{\Sigma}}$.



Corrected: the original writes the covariance estimate with the transpose on the wrong factor, $(\mathbf{x}_i - \boldsymbol{\mu})^T (\mathbf{x}_i - \boldsymbol{\mu})$, which gives a scalar instead of a $d \times d$ matrix. The slide on the EM algorithm writes the same estimate correctly — see [zdroje.md](#).

MLE estimate — by hand on three points

22

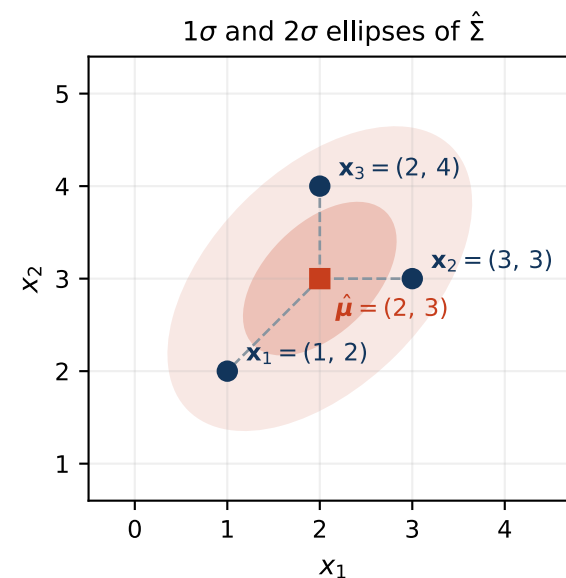
$\mathbf{x}_1 = (1, 2), \mathbf{x}_2 = (3, 3), \mathbf{x}_3 = (2, 4)$, so $\hat{\boldsymbol{\mu}} = \frac{1}{3}(6, 9) = (2, 3)$ and the deviations are $(-1, -1), (1, 0), (0, 1)$.

Outer products $(\mathbf{x}_i - \hat{\boldsymbol{\mu}})(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^T$:

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} + \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \hat{\Sigma} = \frac{1}{3} \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Eigenvalues 1 and $\frac{1}{3}$, eigenvectors $(1, 1)/\sqrt{2}$ and $(1, -1)/\sqrt{2}$ — the ellipse is tilted by 45° and $\sqrt{3}$ times longer along the diagonal.

The wrong order $(\mathbf{x}_i - \hat{\boldsymbol{\mu}})^T(\mathbf{x}_i - \hat{\boldsymbol{\mu}})$ gives $2 + 1 + 1 = 4$, and $\frac{4}{3}$ is a single number — the *trace* of $\hat{\Sigma}$, not a matrix.

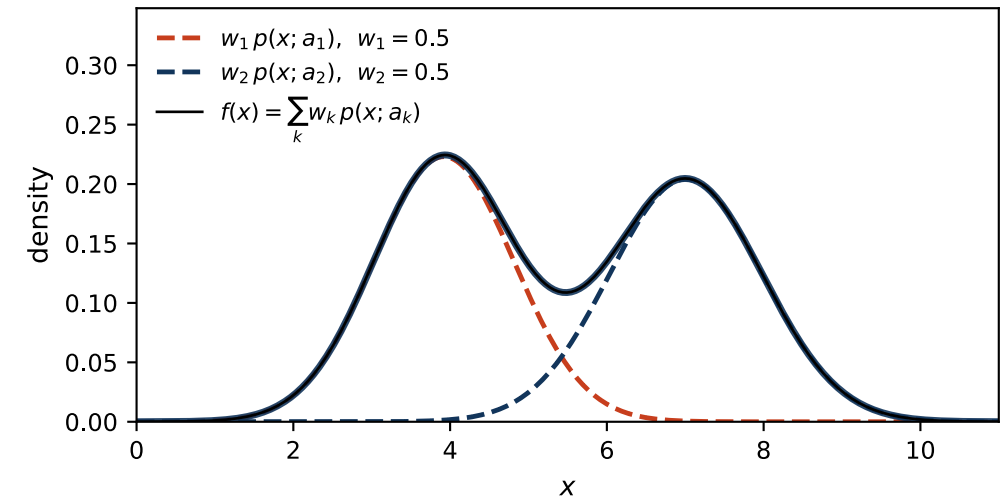


A **mixture model** is a weighted sum of K distributions:

$$f(x; a_1, \dots, a_K, w_1, \dots, w_K) = \sum_{k=1}^K w_k p(x; a_k)$$

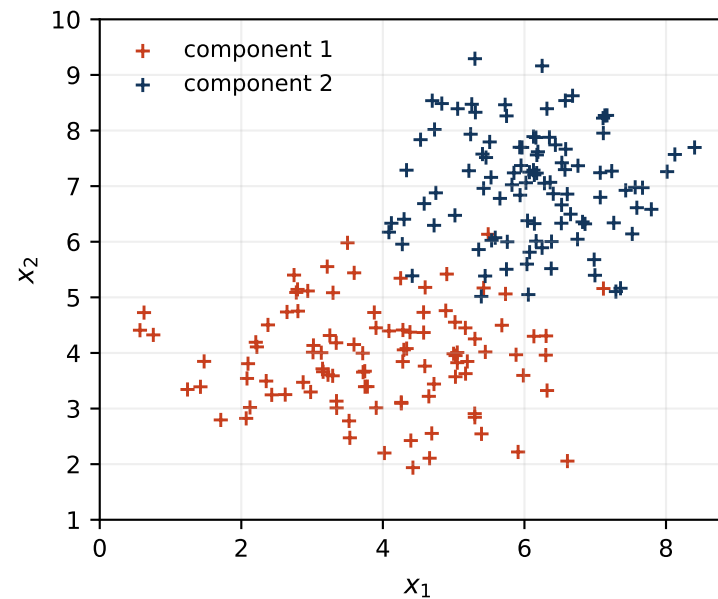
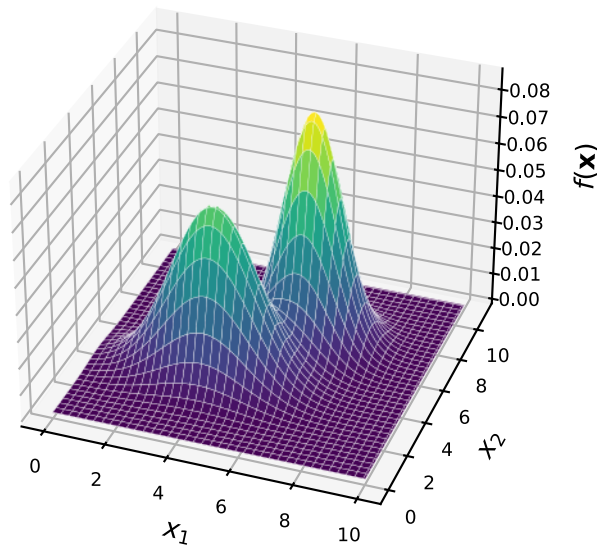
$$\sum_{k=1}^K w_k = 1$$

The weights w_k are the probabilities of the components, so the mixture is a proper density again: it is non-negative and integrates to 1.



$$f(\mathbf{x}; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K, \Sigma_1, \dots, \Sigma_K, w_1, \dots, w_K) = \sum_{k=1}^K w_k p(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$$

$$f(\mathbf{x}) = w_1 p(\mathbf{x}; \boldsymbol{\mu}_1, \Sigma_1) + w_2 p(\mathbf{x}; \boldsymbol{\mu}_2, \Sigma_2)$$



Parameters: K mean vectors, K covariance matrices and K weights.

Here $K = 2, d = 2$, so $2(2 + 3 + 1) - 1 = 11$ free parameters.

GMM estimation — why MLE cannot be used directly

25

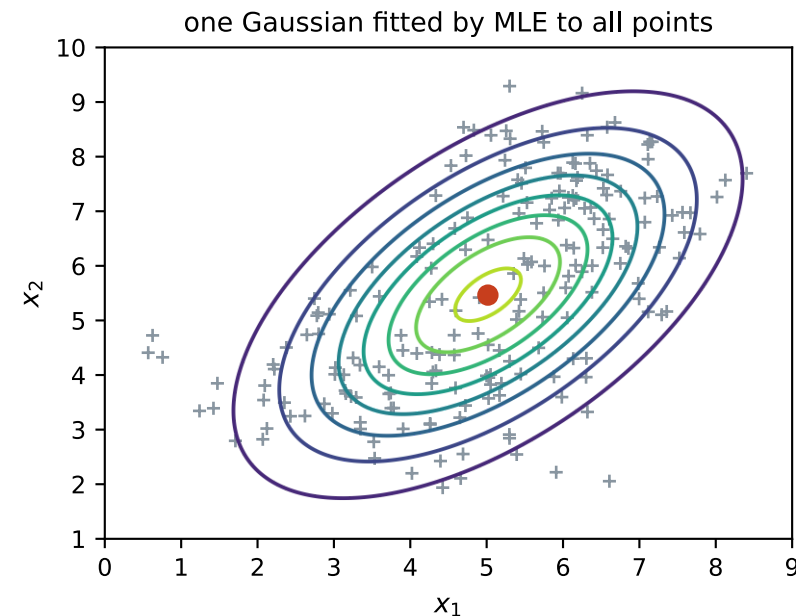
Problem definition for the K -modal case:

$$f(\mathbf{x}; \boldsymbol{\mu}_1, \dots, \boldsymbol{\mu}_K, \Sigma_1, \dots, \Sigma_K, w_1, \dots, w_K) = \sum_{k=1}^K w_k p(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$$

— find the means, covariances and weights of the mixture model.

The closed-form MLE or MAP formulas cannot be used, because the labels are unknown — we do not know which points should be used for which Gaussian.

Therefore the estimation must be solved by **iterative algorithms** (e.g. EM, MCMC).

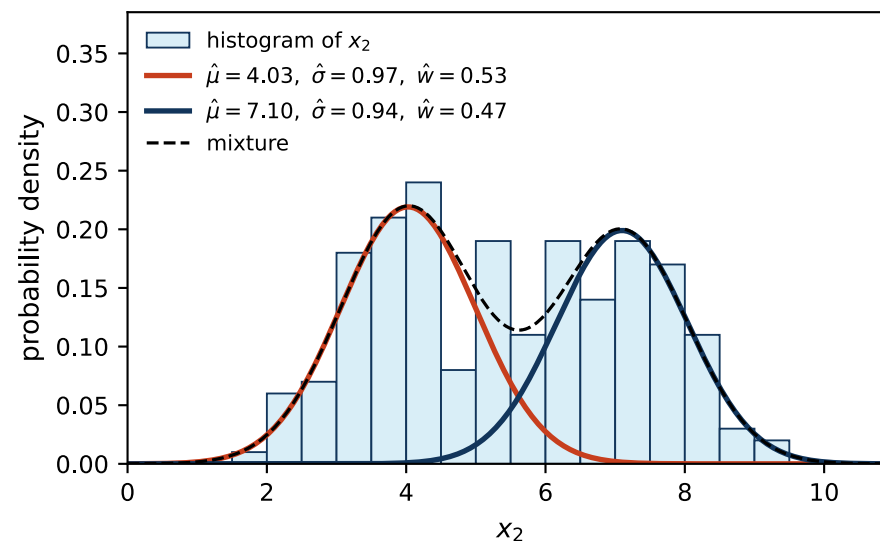
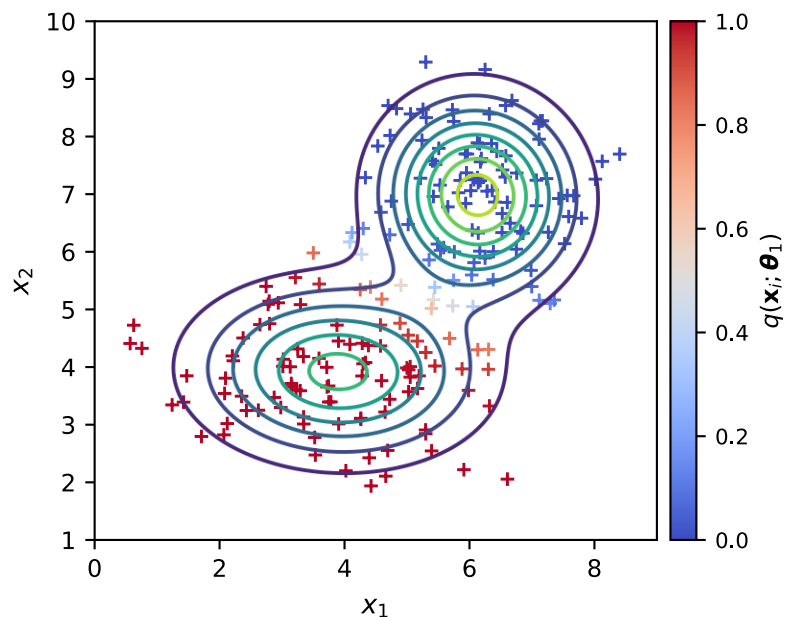


One Gaussian fitted by MLE to all points: the closed-form formulas do work — they just describe neither cluster.

- 1) **Unsupervised clustering of unlabeled data into a known number of classes.** An „improved k -means“ from the first lecture: soft assignments instead of hard ones, and ellipsoidal clusters instead of spherical ones.
- 2) **Approximation of an unknown complex distribution by many Gaussians.**

A probabilistic model that assumes all the data points are generated from a **mixture** of a finite number of Gaussian distributions with unknown parameters. The aim is thus to **estimate the parameters** of those Gaussians.

The original writes „by lot of Gaussians“ and uses Czech quotation marks.

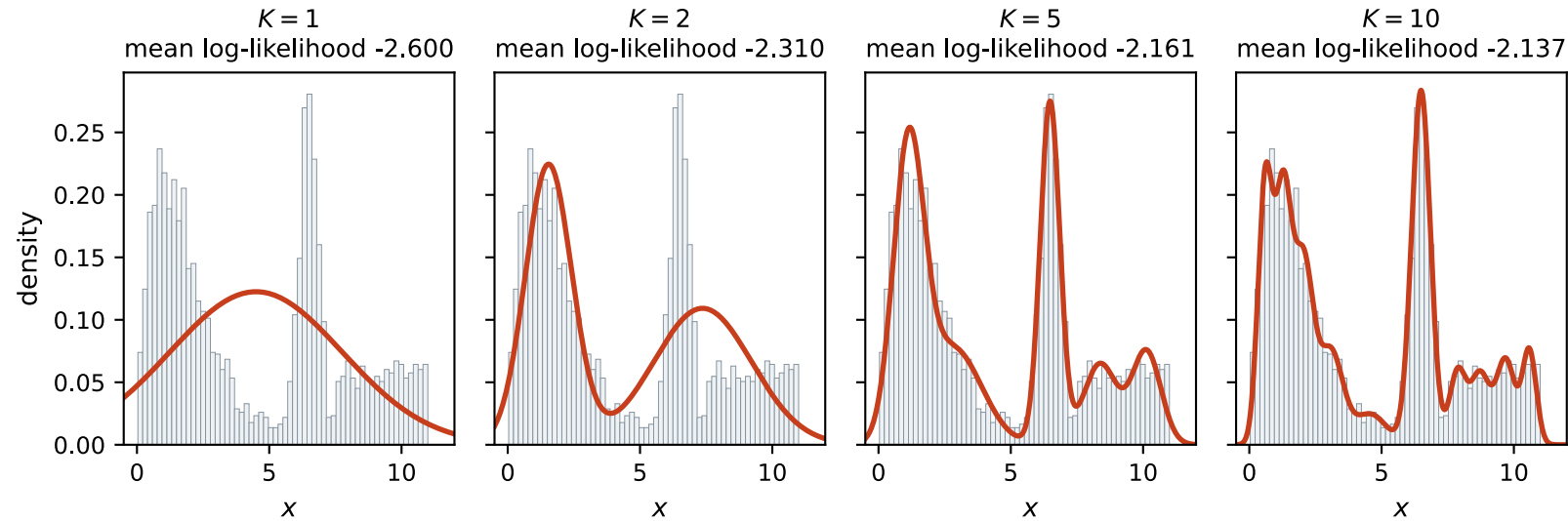


2D clustering into classes — the colour is the responsibility $q(\mathbf{x}_i; \boldsymbol{\theta}_1)$, not a label.

1D fitting of a distribution — the same data, marginal x_2 only; the fit finds $\hat{\mu} = 4.03/7.10, \hat{\sigma} = 0.97/0.94, \hat{w} = 0.53/0.47$.

GMM as a density model — more components, better fit

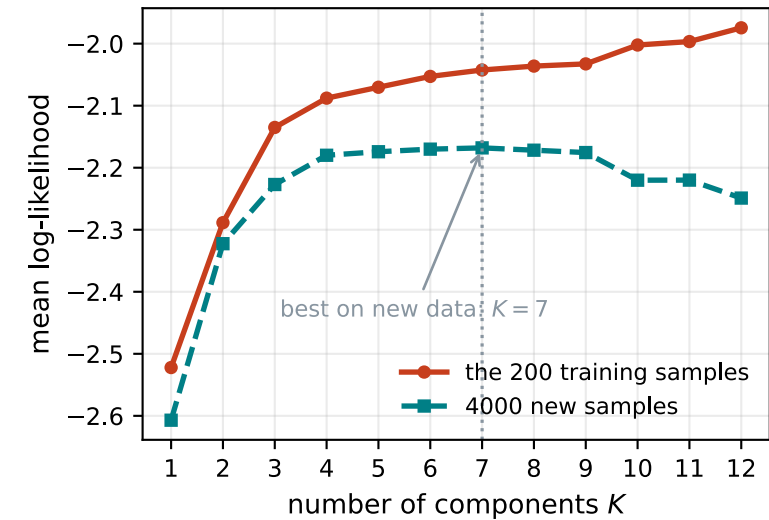
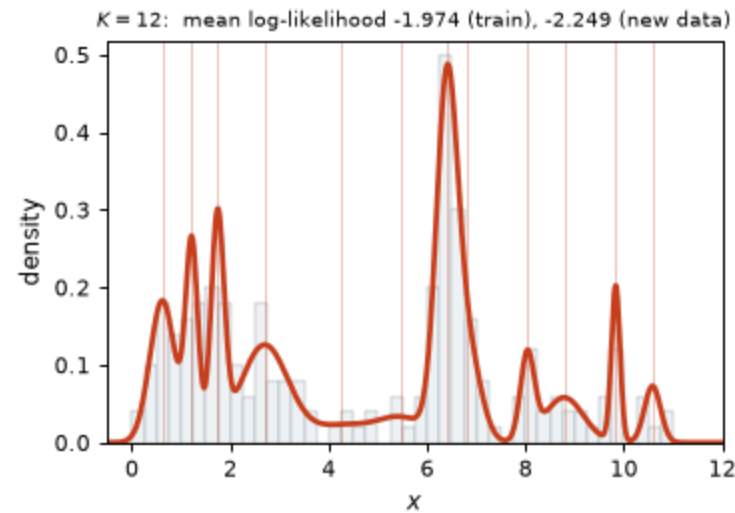
28



The same 4000 samples from a distribution that is **not** Gaussian (skewed, with a narrow peak and a flat tail). With $K = 1$ the model is hopeless; with $K = 10$ it follows the histogram closely. The mean log-likelihood grows from -2.600 to -2.137 — and would keep growing until the model just memorizes the data.

How many components? — the same fit on new data

29



200 of the samples for fitting, **4000 new** samples from the same distribution for checking. On the training samples the mean log-likelihood grows with every extra component (to -1.974 at $K = 12$); on new data it peaks at $K = 7$ (-2.168) and falls to -2.249 at $K = 12$. That is the overfitting from the fifth lecture, only the „model complexity“ is now K .

(Just a note, not important for the exam.)

EM is a general tool in statistics for models with **missing (latent) variables**. Given a model that generates observed data x and missing values z , the likelihood for MLE is a marginal:

$$L(\boldsymbol{\theta}; x) = p(x \mid \boldsymbol{\theta}) = \int p(x, z \mid \boldsymbol{\theta}) dz = \int p(x \mid z, \boldsymbol{\theta}) p(z \mid \boldsymbol{\theta}) dz$$

This integral makes direct maximization intractable. The EM algorithm therefore looks for the MLE of the marginal likelihood by iterating two steps:

Expectation step — compute the expected log-likelihood of $\boldsymbol{\theta}$ under the current estimate $\boldsymbol{\theta}^t$ (an expectation over the missing variables):

$$Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^t) = \mathbb{E}_{z \mid x, \boldsymbol{\theta}^t} [\log L(\boldsymbol{\theta}; x, z)]$$

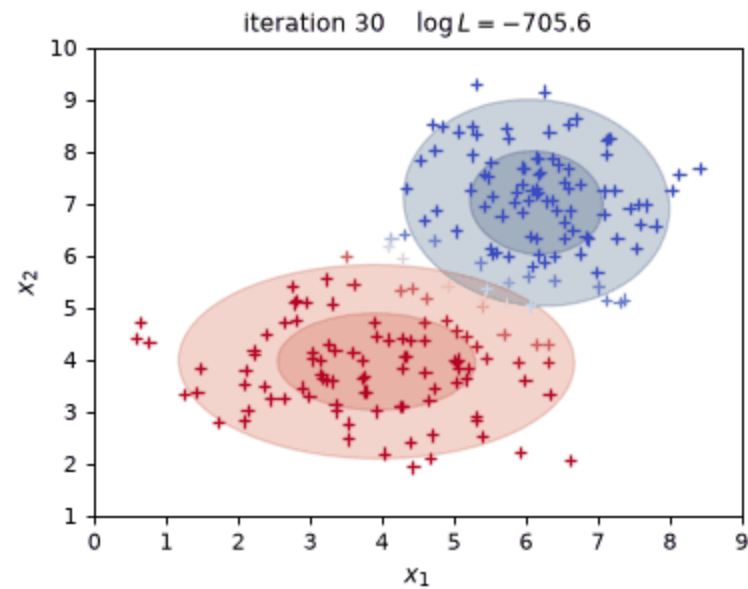
Maximization step — find the parameters that maximize this quantity:

$$\boldsymbol{\theta}^{t+1} = \arg \max_{\boldsymbol{\theta}} Q(\boldsymbol{\theta} \mid \boldsymbol{\theta}^t)$$

Corrected: the original factorizes the joint as $p(z \mid x, \boldsymbol{\theta}) p(x \mid \boldsymbol{\theta})$, which makes the identity circular, and then says that $p(z \mid x, \boldsymbol{\theta})$ *cannot be calculated*. That density is exactly what the E-step computes on the next slide but one; intractable is the **maximization of the marginal likelihood**.

1. Initialize the parameters θ .
2. Expectation (E) step — compute the probability of each possible value of z , given the current estimate of θ .
3. Maximization (M) step — use the estimate of z for a better estimate of θ .
4. Iterate between the E and the M step.

What the two steps do — step by step



The animation starts from the **converged state**, then replays the iterations from the initialization $\mu_1 = \mathbf{x}_1$, $\mu_2 = \mathbf{x}_2$, $\Sigma_k = I$, $w_k = 0.5$.

- **E step** — the colour of every point changes: it is the probability that the point belongs to component 1.
- **M step** — the ellipses move and change shape: each Gaussian is re-estimated from *all* points, weighted by that probability.

Log-likelihood at the start -999.5 , after convergence -705.6 .

E step — evaluate the likelihood of every point $\mathbf{x}_i$ under every Gaussian k and normalize, to get the probability that this point comes from component k (its **responsibility**):

$$q(\mathbf{x}_i; \boldsymbol{\theta}_k) = \frac{w_k p(\mathbf{x}_i; \boldsymbol{\theta}_k)}{\sum_{j=1}^K w_j p(\mathbf{x}_i; \boldsymbol{\theta}_j)}, \quad \boldsymbol{\theta}_k = [\boldsymbol{\mu}_k, \Sigma_k]$$

$$p(\mathbf{x}_i; \boldsymbol{\theta}_k) = (2\pi)^{-\frac{d}{2}} |\Sigma_k|^{-\frac{1}{2}} \exp\left(-\frac{1}{2}(\mathbf{x}_i - \boldsymbol{\mu}_k)^T \Sigma_k^{-1} (\mathbf{x}_i - \boldsymbol{\mu}_k)\right)$$

By construction $\sum_{k=1}^K q(\mathbf{x}_i; \boldsymbol{\theta}_k) = 1$ for every point — it is a distribution over components, one per sample.

This is **Bayes' rule again**: prior w_k , likelihood $p(\mathbf{x}_i; \boldsymbol{\theta}_k)$, posterior q — the same structure as the class posterior in the naive Bayes block, only the „class“ is now hidden.

M step — take the MLE estimate for each Gaussian, with every point weighted by the probability that it belongs to component k :

$$w_k = \frac{1}{n} \sum_{i=1}^n q(\mathbf{x}_i; \boldsymbol{\theta}_k), \quad \boldsymbol{\mu}_k = \frac{\sum_{i=1}^n q(\mathbf{x}_i; \boldsymbol{\theta}_k) \mathbf{x}_i}{\sum_{i=1}^n q(\mathbf{x}_i; \boldsymbol{\theta}_k)}$$
$$\Sigma_k = \frac{\sum_{i=1}^n q(\mathbf{x}_i; \boldsymbol{\theta}_k) (\mathbf{x}_i - \boldsymbol{\mu}_k) (\mathbf{x}_i - \boldsymbol{\mu}_k)^T}{\sum_{i=1}^n q(\mathbf{x}_i; \boldsymbol{\theta}_k)}$$

These are the **same MLE formulas** as for a single Gaussian, with the counts replaced by sums of responsibilities. If all q were 0 or 1, they would reduce to „split the points by label and fit each part separately“.

[kod/em_gmm.py](#) — both steps in about 30 lines of numpy, checked against `sklearn.mixture.GaussianMixture`.

GMM estimation by EM — the algorithm

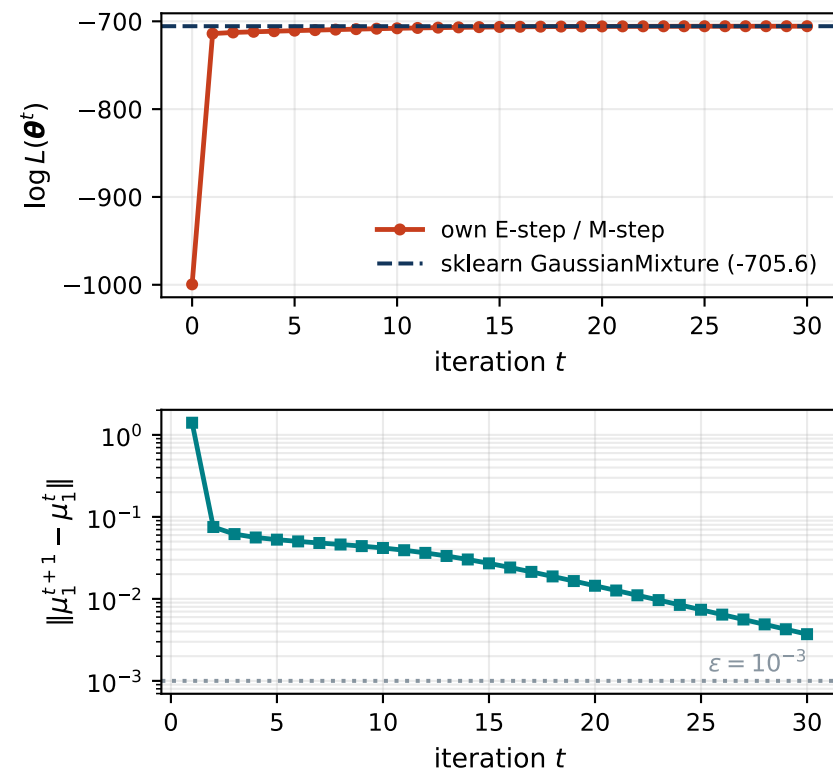
35

Algorithm for two 2D Gaussians ($d = 2, K = 2$):

1. Initialization of $\mu_1, \mu_2, \Sigma_1, \Sigma_2, w_1, w_2$.
2. Compute the normalized likelihoods $q(\mathbf{x}_i; \theta_1)$ and $q(\mathbf{x}_i; \theta_2)$ for all points (**E step**).
3. Compute w_1 and w_2 as the mean of $q(\mathbf{x}_i; \theta_k)$ over all points, and re-estimate $\mu_1, \mu_2, \Sigma_1, \Sigma_2$ (**M step**).
4. Repeat steps 2–3 **until** a stopping criterion is met.

Stopping criterion:

- maximal number of iterations,
- change of the parameters below a limit, $\|\mu^{t+1} - \mu^t\| < \epsilon$,
- (in practice also: increase of $\log L$ below a limit).



3 · Logistic regression

The promise from the fourth lecture, now with the derivation:
where the sigmoid and the log-loss actually come from.

Logistic regression is a method for building a statistical model used in **classification problems**. It lets us take some features and predict the correct class — or better, the **probability of that class**.

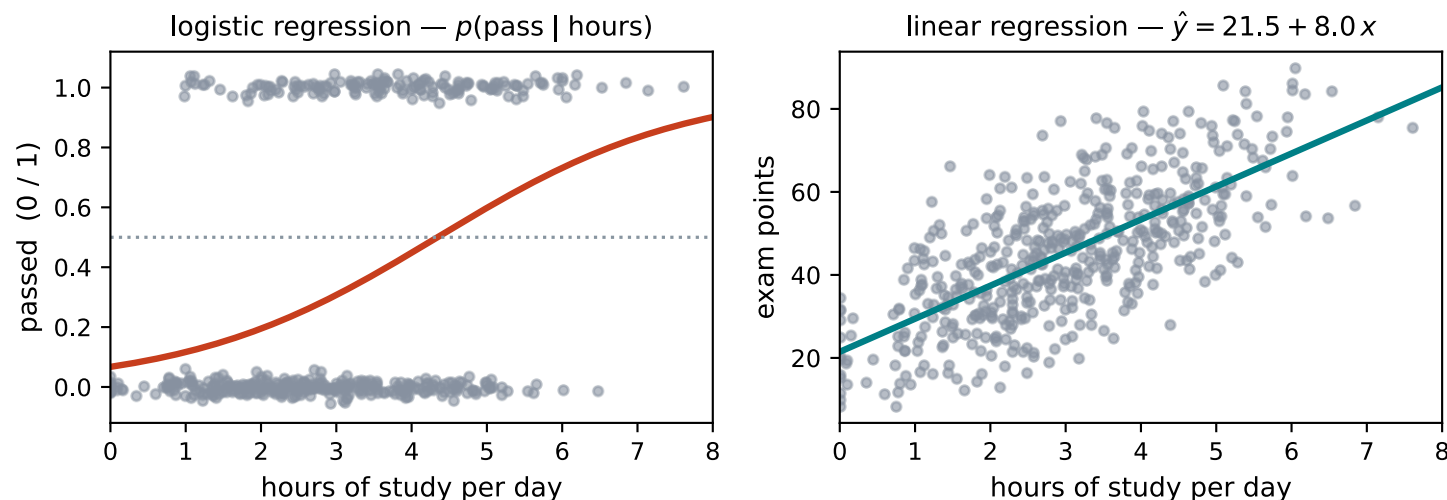
We met it already in the fourth lecture, as a linear classifier with a sigmoid and a log-loss. Today we derive it: the sigmoid and the loss both fall out of a **Bernoulli likelihood plus MLE**.

Despite the name, it is a **classification** method. It is *discriminative* — it models $p(y \mid \mathbf{x})$ directly and never models $p(\mathbf{x})$, unlike the naive Bayes classifier from the first block.

Linear versus logistic regression

38

- We can predict **whether** a student passes the exam (yes / no) based on the number of hours spent studying. That is a task for **logistic regression**.
- We can predict **how many points** the student gets („almost“ a continuous variable) based on the number of hours spent studying. That is a task for **linear regression**.

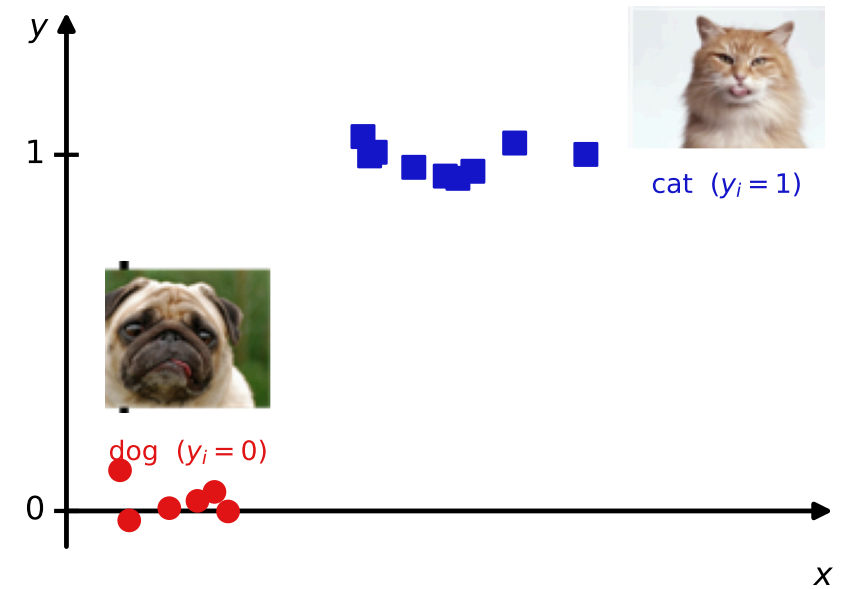


The same input, two different outputs — and therefore two different models. The sigmoid on the left cannot leave $[0, 1]$; the line on the right can predict any number. The 500 students are the seminar data set; the exam score on the right is **added** for this comparison, because the file records only pass/fail.

We want to predict the probability that we classify a **cat** (i.e. $y_i = 1$), given the input (a feature vector $\mathbf{x}_i$) and the parameters of the model $\mathbf{w}$:

$$p(y_i = 1 \mid \mathbf{x}_i, \mathbf{w}) = ?$$

It does not make sense to use a **linear** model, because it would just fit a line through our data. But we can use some function to transform the linear model into something more convenient.



Sigmoid function

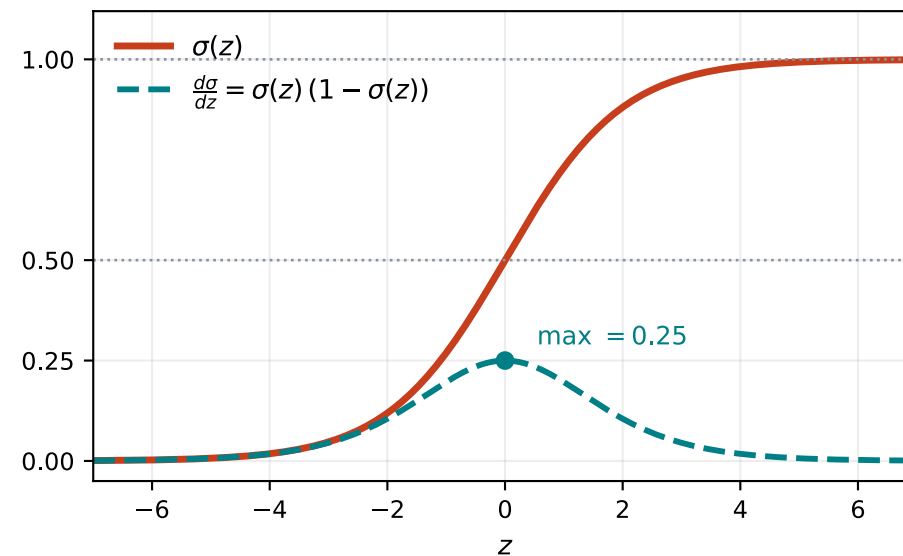
For this purpose we use the **sigmoid** function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Its useful properties:

- whatever the input is, it is transformed into $(0, 1)$,
- the derivative with respect to the input is very convenient:

$$\frac{d\sigma(z)}{dz} = \sigma(z) (1 - \sigma(z))$$



The derivative never exceeds 0.25 (for the plain sigmoid used here; the sixth lecture writes it with a temperature T , where the maximum is $1/(4T)$) – hence the vanishing-gradient problem.

We use the sigmoid to transform the linear model:

$$\theta = \sigma(\mathbf{w}^T \mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T \mathbf{x}}}$$

Bernoulli probability distribution (from the previous lecture):

$$f(y \mid \theta) = \theta^y (1 - \theta)^{1-y}, \quad y \in \{0, 1\}$$

We can now look at our data in a different way: the **labels are Bernoulli random variables**, and the model given by the sigmoid supplies their parameter θ — a different one for every sample.

Notation: here $\mathbf{w}^T \mathbf{x}$ has the **bias absorbed into the weights** — $x_0 = 1$ and $w_0 = b$. The fourth lecture writes the same model as $\mathbf{w}^T \mathbf{x} + b$; the two are identical.

The likelihood is the „probability“ that the observed labels come from a specific model (Bernoulli) with parameters $\mathbf{w}$. For **independent and identically distributed** samples it is a product:

$$\mathcal{L}(\mathbf{w} \mid \mathbf{y}, \mathbf{x}_1, \dots, \mathbf{x}_n) = \prod_{i=1}^n \sigma(\mathbf{w}^T \mathbf{x}_i)^{y_i} (1 - \sigma(\mathbf{w}^T \mathbf{x}_i))^{1-y_i}$$

For a sample with $y_i = 1$ only the first factor survives, for $y_i = 0$ only the second — the exponents work as a switch.

The original writes $\prod_i^n$ without the lower limit, and describes the likelihood as the probability of the data x ; formally it is the likelihood of the **labels $\mathbf{y}$** given the inputs — which the formula itself has right.

Solution — the log-likelihood is the negative cross-entropy

Maximizing a product is numerically bad. Better to maximize the **log** of the likelihood:

$$\mathcal{L}_{\log} = \log \mathcal{L}(\mathbf{w} \mid \mathbf{y}, \mathbf{x}_1, \dots, \mathbf{x}_n) = \sum_{i=1}^n y_i \log(\sigma(\mathbf{w}^T \mathbf{x}_i)) + (1 - y_i) \log(1 - \sigma(\mathbf{w}^T \mathbf{x}_i))$$

With $\hat{y}_i = \sigma(\mathbf{w}^T \mathbf{x}_i)$ this is exactly the **binary cross-entropy of the fourth lecture** (the two-class case of the weighted cross-entropy of the sixth), with the opposite sign and averaged over the n samples:

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] = -\frac{1}{n} \mathcal{L}_{\log}$$

Maximizing the log-likelihood is minimizing the binary cross-entropy. Computed on the student data with $n = 500$:

- $\mathcal{L}_{\log} = -148.78$
- $\text{BCE} = 0.297569$
- $-n \cdot \text{BCE} = -148.78$

Corrected: the original says the log-likelihood „is binary cross-entropy“. It is its **negative**; cross-entropy is defined with the minus sign in the sixth lecture and is *minimized*, while the log-likelihood is *maximized*. Careful with the symbol: the fourth lecture writes $\mathcal{L}_{\log}$ for the *log-loss*, i.e. for $-\mathcal{L}_{\log}$ of this slide.

Solution — the derivative and gradient ascent

We maximize with respect to $\mathbf{w}$, so we take the derivative. Thanks to $\sigma' = \sigma(1 - \sigma)$ almost everything cancels:

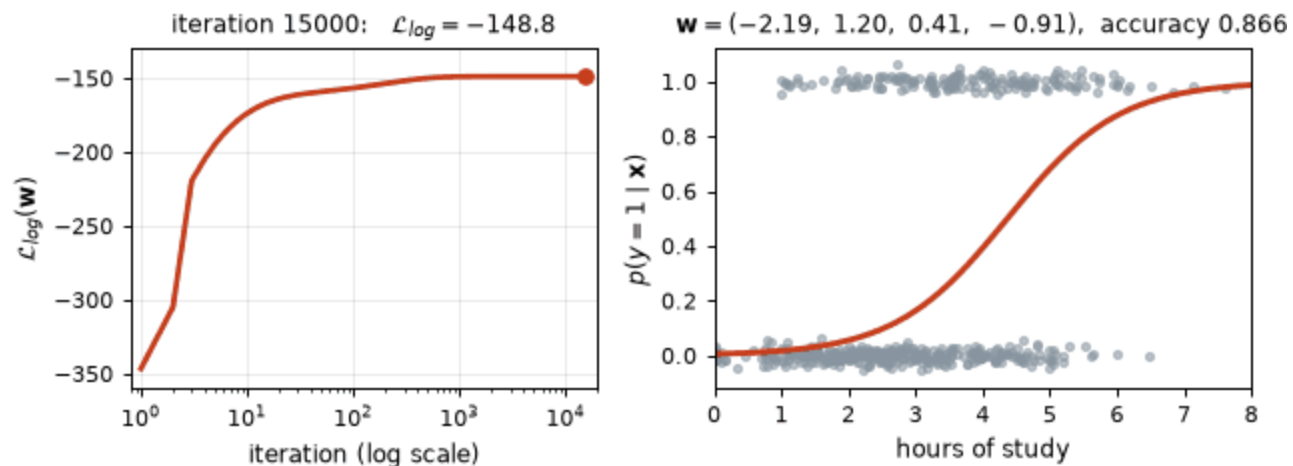
$$\frac{d \mathcal{L}_{\log}(\mathbf{w} \mid \mathbf{y}, \mathbf{x}_1, \dots, \mathbf{x}_n)}{d w^{(j)}} = \sum_{i=1}^n (y_i - \sigma(\mathbf{w}^T \mathbf{x}_i)) x_i^{(j)}$$

There is no closed-form solution, so we optimize numerically — **gradient ascent**:

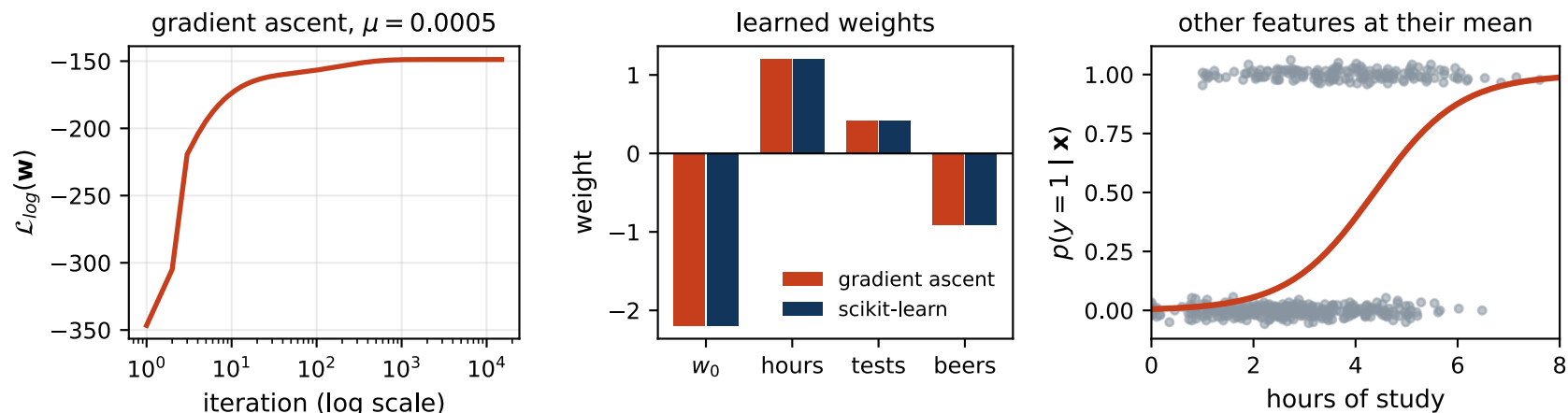
$$w_{\text{new}}^{(j)} = w_{\text{old}}^{(j)} + \mu \sum_{i=1}^n (y_i - \sigma(\mathbf{w}^T \mathbf{x}_i)) x_i^{(j)}$$

The gradient is the **error** $y_i - \hat{y}_i$ times the input — the same form as the weight update derived in the sixth lecture.

The original writes an extra comma right after the conditioning bar, $\mathcal{L}(\mathbf{w} \mid, \mathbf{y}, \dots)$ — twice, in the derivative and in the log-likelihood above it.



Every frame is one state of $\mathbf{w}$ during the update above, starting from $\mathbf{w} = \mathbf{0}$ — where the model says $p = 0.5$ for everybody and $\mathcal{L}_{\log} = n \log \frac{1}{2} = -346.6$. The first few hundred iterations do almost all the work; the remaining thousands only sharpen the curve.



500 students, three features (hours of study, practice tests, beers the day before), trained by the gradient ascent formula above with $\mu = 0.0005$.

	w_0	hours	tests	beers
gradient ascent	-2.191	1.197	0.411	-0.910
generating model	-2.0	1.2	0.4	-0.9

Accuracy on those same 500 students 0.866; the largest difference from `sklearn.linear_model.LogisticRegression` without regularization is 0.0003. [kod/logisticka_regrese.py](#)

Interpretation — odds and log-odds

47

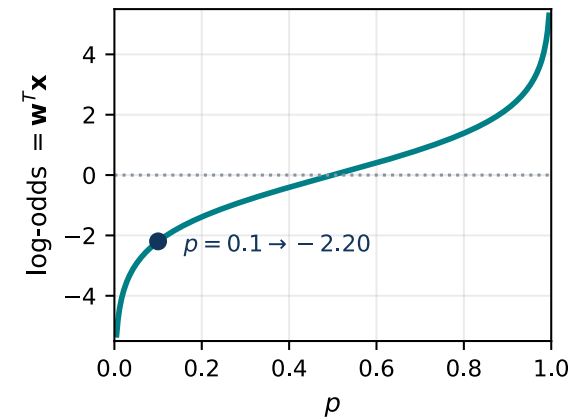
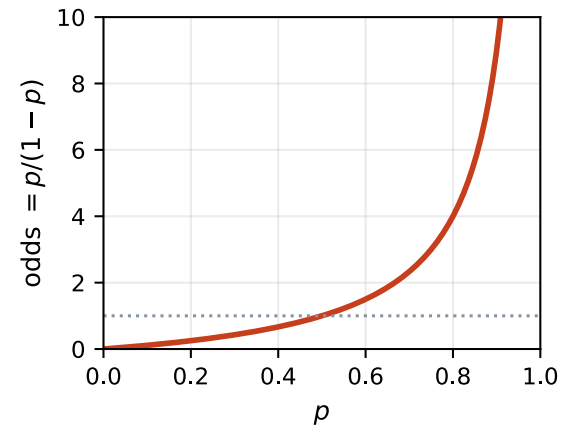
Odds = $\frac{p}{1-p}$ — e.g. $\frac{0.2}{0.8} = 0.25$ for a probability of 20 %.

Log-odds are what the linear part of the model predicts:

$$\log \frac{p}{1-p} = \mathbf{w}^T \mathbf{x}$$

A single weight w_j is therefore read like a weight in linear regression:

- large positive $\rightarrow$ increasing this feature leads to a larger probability,
- large negative $\rightarrow$ increasing this feature leads to a smaller probability,
- close to 0 $\rightarrow$ an unimportant feature.



Odds ratio — e to the power of the weight, $OR_j = e^{w_j}$. If the corresponding feature increases by 1, the **odds are multiplied** by the odds ratio.

OR_j		starting p	starting odds	new odds	new p
1.42 (increase by 42 %)		10 %	0.1111	0.1578	13.6 %
0.42 (decrease to 42 %)		10 %	0.1111	0.0467	4.5 %

From the trained model of the previous slide:

feature	w_j	OR_j	10 % becomes
hours of study	1.197	3.309	26.9 %
practice tests	0.411	1.509	14.4 %
beers	−0.910	0.402	4.3 %

The original stops at the new odds; the last column (the new probability) is computed here, because that is the number a reader understands.

Summary — three models, one recipe

49

	naïve Bayes	Gaussian mixture	logistic regression
task	classification	clustering / density	classification
type	generative	generative (no classes)	discriminative
models	$p(\mathbf{x} \mid y_k) p(y_k)$	$p(\mathbf{x})$	$p(y \mid \mathbf{x})$
labels needed	yes	no	yes
assumption	features independent given the class	data are a mixture of K Gaussians	log-odds are linear in $\mathbf{x}$
estimation	MLE / MAP, closed form	EM, iterative	gradient ascent on $\mathcal{L}_{\log}$
output	posterior over K classes	responsibilities q	probability of class 1

Every one of them is the same recipe: **write down the likelihood, then maximize it.**

Summary — where the loss functions came from

50

assumption about the data	maximum likelihood gives	known from
Gaussian noise $\varepsilon \sim \mathcal{N}(0, \sigma^2)$	L2 loss $\sum_i (y_i - \hat{y}_i)^2$	sixth lecture
Bernoulli labels $y \sim \text{Bern}(\sigma(\mathbf{w}^T \mathbf{x}))$	binary cross-entropy	fourth and sixth lecture
Gaussian prior on the weights	L2 regularization / weight decay	sixth lecture
a mixture of K Gaussians	the EM algorithm	today

Loss functions are not arbitrary. Each of them is — $\log$ of a probabilistic assumption — which is why probability theory is the language in which new models are invented.

4 · Recap

Three sets of questions: the Bayes classifier, the mixture model, and logistic regression.

- Write **Bayes' rule for a class posterior** and name all four terms. Which one can be dropped when we only want the most probable class, and why?
- Derive the **L2 loss** from maximum likelihood. Which assumption about the noise did you use, and what would change for Laplace noise?
- State the **naive assumption**. Is it independence of the features, or something weaker?
- How many distributions does a naive Bayes classifier fit for K classes and d features? How many parameters does one full d -dimensional Gaussian need?
- Naive Bayes is called *robust to missing features*. Explain why, using the product formula.
- The naive assumption is almost always violated. Why does the classifier still work?

- Write the density of a **multivariate Gaussian**. What shape must Σ have, and what would go wrong if the off-diagonal entries were $\sigma_i \sigma_j$?
- Write the MLE estimate of Σ . Which factor carries the transpose, and what is the shape of the result?
- Why **cannot** the closed-form MLE formulas be used for a mixture? What exactly is missing?
- Write the **E step** and say which quantity from the first block it is the same as. For one point, $w_1 p(\mathbf{x}_i; \boldsymbol{\theta}_1) = 0.03$ and $w_2 p(\mathbf{x}_i; \boldsymbol{\theta}_2) = 0.01$ — what are the two responsibilities?
- Write the **M step**. How does it differ from fitting a single Gaussian?
- What does the **log-likelihood** do from iteration to iteration, and what follows for the stopping criterion?
- How is a GMM related to **k-means** from the first lecture? Name two differences.

- Why does a **linear** model not work for a binary label? What exactly does the sigmoid fix?
- Write $\sigma(z)$ and its derivative. What is the maximum of the derivative, and where did we meet that number before?
- Write the **likelihood** of logistic regression. Where does the Bernoulli distribution enter?
- Show that the log-likelihood is the **negative binary cross-entropy**. Which one do we maximize and which one do we minimize?
- Write the gradient ascent update. Why is there a plus and not a minus?
- A weight is $w_j = 0.7$. What is the odds ratio, and what does a probability of 20 % become if that feature increases by 1?
- Which of today's three models is **discriminative** and which are **generative**? What can a generative model do that a discriminative one cannot?